

NimbusPay

Documentación Técnica Interna
Stack Tecnológico, Arquitectura e Infraestructura

Versión 3.2 — Actualizado en junio de 2026
Uso interno — Equipo de Ingeniería

1. Introducción

NimbusPay es una fintech que ofrece servicios de pagos digitales y billetera electrónica para pequeñas y medianas empresas en Latinoamérica. Este documento describe el stack tecnológico utilizado por los equipos de Ingeniería, las políticas de despliegue y los estándares de seguridad vigentes.

2. Arquitectura de Backend

La plataforma de pagos de NimbusPay está construida sobre una arquitectura de microservicios. El lenguaje principal del back-end es Python, utilizando el framework FastAPI para exponer las APIs REST internas y externas. Los servicios de procesamiento de transacciones de alto volumen, como el motor de conciliación bancaria, están escritos en Go por razones de rendimiento y concurrencia.

La comunicación asíncrona entre microservicios se realiza mediante Apache Kafka, utilizado principalmente para eventos de transacciones, notificaciones y auditoría. Para tareas programadas y colas de trabajo se utiliza Celery junto con Redis como broker.

2.1 Bases de datos

Servicio	Base de datos	Motivo
Core de transacciones	PostgreSQL 15	Consistencia ACID, transacciones críticas
Catálogo de comercios	PostgreSQL 15	Relacional, consultas complejas
Sesiones y caché	Redis 7	Baja latencia, expiración automática
Logs de auditoría	MongoDB	Alto volumen, esquema flexible
Búsqueda de comercios	Elasticsearch	Búsqueda full-text y geolocalización

3. Arquitectura de Frontend

El panel web para comercios (NimbusPay Dashboard) está desarrollado en React 18 con TypeScript, utilizando Vite como bundler y TailwindCSS para el sistema de diseño. La aplicación móvil, disponible para iOS y Android, está construida con React Native, compartiendo gran parte de la lógica de negocio con el dashboard web a través de una librería interna llamada nimbus-core.

Para el manejo de estado global se utiliza Zustand, y las peticiones a la API se gestionan con React Query, lo que permite cacheo automático y reintentos ante fallos de red.

4. Infraestructura y Despliegue

Toda la infraestructura de NimbusPay corre sobre Oracle Cloud Infrastructure (OCI). Los microservicios se despliegan como contenedores Docker orquestados con Kubernetes (OKE - OCI Kubernetes Engine). El pipeline de integración y despliegue continuo (CI/CD) utiliza GitHub Actions para las pruebas

automáticas y GitLab CI para los despliegues a producción.

Los despliegues a producción siguen una estrategia de blue-green deployment, minimizando el tiempo de inactividad. Cada despliegue requiere la aprobación de al menos un ingeniero senior y debe pasar la suite completa de pruebas automatizadas, que incluye pruebas unitarias, de integración y de carga.

4.1 Monitoreo y observabilidad

El monitoreo de servicios se realiza con Prometheus y Grafana para métricas, y con el stack ELK (Elasticsearch, Logstash, Kibana) para centralización de logs. Las alertas críticas se envían a través de PagerDuty, integrado con el canal de Slack #incidentes-produccion.

5. Seguridad y Cumplimiento

Por ser una fintech, NimbusPay cumple con el estándar PCI DSS nivel 1. Toda la información sensible de tarjetas se tokeniza mediante un proveedor externo certificado y nunca se almacena en las bases de datos internas. La autenticación de usuarios utiliza OAuth 2.0 con tokens JWT de corta duración (15 minutos) y refresh tokens rotativos.

Todo el tráfico entre servicios internos está cifrado con mTLS (mutual TLS), y el acceso a los servidores de producción requiere autenticación multifactor (MFA) además de una VPN corporativa gestionada por el equipo de Seguridad.

6. Herramientas de Inteligencia Artificial

El equipo de Datos utiliza modelos de machine learning para la detección de fraude en tiempo real, implementados en Python con scikit-learn y XGBoost, servidos a través de un endpoint interno construido con FastAPI. Desde 2025, el equipo de soporte al cliente utiliza un asistente conversacional basado en modelos de lenguaje (LLM) de Google Gemini para responder consultas frecuentes de los comercios, integrado directamente en el dashboard web.

7. Roadmap Tecnológico 2026

- Migración del catálogo de comercios de PostgreSQL a una arquitectura particionada para soportar más de 10 millones de registros.
- Adopción de gRPC para la comunicación entre microservicios de alta frecuencia, reemplazando gradualmente REST interno.
- Implementación de un data lake sobre OCI Object Storage para analítica avanzada con Apache Spark.
- Certificación ISO 27001 prevista para el cuarto trimestre de 2026.